

Федеральное государственное автономное образовательное учреждение высшего образования
Санкт-Петербургский Политехнический Университет Петра Великого
Физико-Механический Институт

Курсовая работа по предмету «Методы оптимизации»
**«Решение обобщённой задачи
коммивояжёра для оптимизации холостого
хода станков с ЧПУ»**

Преподаватель:

Выполнили студенты гр. 5030102/10401:

Дата:

Родионова Елена Александровна

Безлепский Александр Дмитриевич

Прохоров Артём Дмитриевич

29 октября 2024 г.

Содержание

1	Введение	3
1.1	Общие сведения	3
1.2	Классификация задач оптимальной маршрутизации	3
1.3	Цель работы	4
2	Подходы к решению обобщённой задачи коммивояжёра	4
2.1	Точные алгоритмы	4
2.2	Методы трансформации	5
2.3	Алгоритмы сокращения	5
2.4	Эвристические алгоритмы	5
3	Формализация обобщённой задачи коммивояжёра	5
4	Описание использованных алгоритмов	7
4.1	Алгоритм полного перебора	7
4.2	Простейшая эвристика: жадный алгоритм	8
4.3	Точный алгоритм на основе динамического программирования	8
5	Тестовый пример	10
5.1	Описание тестовой задачи	10
5.2	Решение тестового примера жадным алгоритмом	11
5.3	Решение тестового примера методом динамического программирования	12
6	Сравнение методов решения обобщенной задачи коммивояжёра	16
7	Заключение	18
7.1	Заключение	18
7.2	Вывод	18

1 Введение

1.1 Общие сведения

В современных производственных процессах, связанных с обработкой изделий на лазерных станках с числовым программным управлением (**ЧПУ**, Computer Numerical Control, **CNC**), одной из ключевых задач является построение маршрута лазерной головки. В частности, для лазерных станков с ЧПУ, использующихся для резки металлических и неметаллических листовых деталей, существенное значение имеет минимизация холостого хода инструмента, то есть перемещения его без реза. Это позволяет не только сократить общее время обработки деталей, но и уменьшить износ оборудования.

1.2 Классификация задач оптимальной маршрутизации

Задача минимизации холостого хода нетривиальна, требует учета множества факторов, таких как геометрия обрабатываемых контуров, их расположение на рабочей плоскости, а также технические ограничения используемого оборудования. В простейшем случае она может быть сведена к поиску кратчайшей ломаной с вершинами на заданных плоских контурах. В научной литературе при описании алгоритмов решения задач маршрутизации инструмента для машин листовой резки с ЧПУ рассматриваются следующие классы задач [4]:

1. **Задача коммивояжера** (Traveling Salesman Problem, TSP). Разрезание каждого контура начинается и заканчивается в единственной заранее определенной на нём точке. Деталь вырезается целиком. Применяется для задач оптимизации порядка обхода деталей;
2. **Обобщенная задача коммивояжера** (Generalized Traveling Salesman Problem, GTSP). Резка каждого контура может начаться в одной из заранее заданного конечного множества точек, после этого контур вырезается целиком;
3. **Задача резки с конечным набором точек** (Endpoint Cutting Problem, ECP). Резка контура начинается в заранее заданных точках (из конечного множества) на нём, но не обязательно вырезать всю деталь за один проход. Применяется, когда точки входа/выхода предопределены технологическим процессом;
4. **Задача непрерывной резки** (Continuous Cutting Problem, CCP). Резка может начаться в любой точке контура (из непрерывного множества) и должна в ней закончиться. Детали вырезаются целиком;
5. **Сегментная задача непрерывной резки** (Segment Continuous Cutting Problem, SCCP): вводится понятие сегмента как обобщение понятия контура; сегмент может быть частью контура или объединением нескольких контуров и (или) их частей. Каждый сегмент вырезается целиком, от начала до конца, таким образом $CCP \subset SCCP$;
6. **Обобщённая сегментная задача непрерывной резки** (Generalized Segment Continuous Cutting Problem, GSCCP): Подобна сегментной задаче непрерывной резки (SCCP), но разбивка на сегменты не задана заранее, и сама подлежит оптимизации.

7. **Задача произвольной резки** (Intermittent Cutting Problem, ICP). Общий случай задачи маршрутизации, когда не накладывается никаких ограничений на выбор точек начала и конца резки, а также на последовательность резки контуров и их частей.

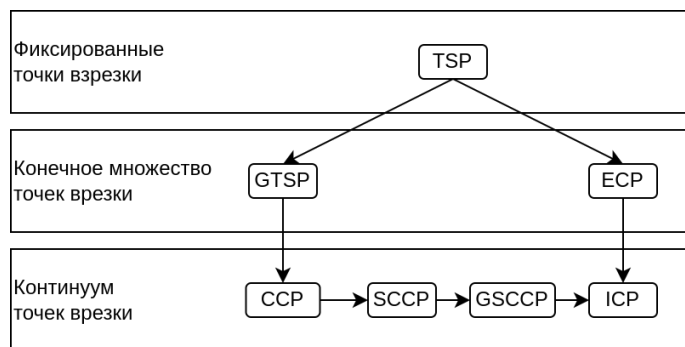


Рис. 1: Схема классификации задач резки

Основное различие между этими классами задач заключается в правилах выбора возможных точек/областей взрезки и разрешении/запрете на частичное отрезание детали.

1.3 Цель работы

Цель данной курсовой работы – постановка обобщённой задачи коммивояжёра (GTSP) на плоскости в контексте оптимизации холостого хода головки лазерных станков с ЧПУ; анализ эффективности точных (полный перебор, динамическое программирование) алгоритмов и эвристического жадного алгоритма, оценить их применимость для задач с различными параметрами (количество кластеров, количество точек в кластерах); определение подходов к решению задачи GTSP в зависимости от размера и сложности входных данных.

2 Подходы к решению обобщённой задачи коммивояжёра

Рассмотрим некоторые подходы к решению обобщённой задачи коммивояжёра [5]:

2.1 Точные алгоритмы

Точные методы решения задачи GTSP обеспечивают получение оптимального решения. Однако, поскольку задача GTSP является NP-трудной, это означает, что не существует известных алгоритмов для её решения за полиномиальное время, и вычислительная сложность этих методов (сверх)экспоненциально возрастает с увеличением числа точек и кластеров. Это делает их применение неэффективным для больших экземпляров. К числу таких методов относятся:

- Полный перебор решений. Данный подход целесообразно использовать только для задач с малым числом кластеров и точек.
- Динамическое программирование (ДП) помогает решать сложные задачи, разбивая их на множество мелких и постепенно переходя к более крупным решениям. Существует два подхода к ДП, которые отличаются тем, с какой стороны задачи они начинают решение.

- Метод «сверху-вниз» (или мемоизация) позволяет экономить вычисления за счёт того, что мы решаем каждую подзадачу только один раз. Как только находим ответ для подзадачи, мы запоминаем его и используем повторно, если она встретится снова.
- Метод «снизу-вверх» более структурированный: вместо прямого решения мы строим цепочку решений для более простых подзадач, постепенно доходя до полной задачи. Так подход помогает найти ответ с минимальным количеством повторных операций.

- Метод ветвей и границ [6].

2.2 Методы трансформации

Преобразование GTSP в классический TSP позволяет применять эффективные методы для TSP к GTSP. Основные подходы:

- Преобразование обобщённой задачи коммивояжёра в обычную задачу коммивояжёра [7]
- Трансформация Нуна и Бина [3]

2.3 Алгоритмы сокращения

Среди которых

- Методы Гутина и Карапетяна [2] – удаляют вершины и рёбра графа задачи, не влияя на оптимальность решения
- Метод предобработки Эль Крари и Бенани [1] – для каждого кластера оставляет лишь точки, ближайшие к другим кластерам; в тестах показано уменьшение количества точек задачи вплоть до 2% от исходного.

2.4 Эвристические алгоритмы

Используют приближенные методы для быстрого нахождения решений без гарантии оптимальности.

3 Формализация обобщённой задачи коммивояжёра

- Заданы положительные числа $0 < N \leq K$, $N, K \in \mathbb{N}$:
- На плоскости заданы K точек:

$$\mathcal{P} = \{p_i = (x_i, y_i) \in \mathbb{R}^2\}, \quad i \in \{1, \dots, K\}$$

- Заданы N положительных чисел, представляющих размеры кластеров:

$$\mathcal{A} = \{a_i \in \mathbb{N} : a_i > 0\}, \quad i \in \{1, \dots, N\}$$

с условием, что сумма размеров кластеров равна общему числу точек:

$$\sum_{i=1}^N a_i = K$$

- Точки разбиты на N непересекающихся кластеров C_i , где $|C_i| = a_i$:

$$\mathcal{C} = \{C_i \subset \mathcal{P} : |C_i| = a_i\}, \quad i \in \{1, \dots, N\}$$

$$\bigcup_{i=1}^N C_i = \mathcal{P}$$

$$C_i \cap C_j = \emptyset \quad \forall i \neq j, \quad i, j \in \{1, \dots, N\}$$

- Задана функция d , вычисляющая евклидово расстояние:

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}, \quad p_i, p_j \in \mathbb{R}^2$$

- Задана стартовая точка: $p_0 = (x_0, y_0) \in \mathbb{R}^2$
- Маршрут (путь обхода кластеров, посещающий каждый кластер ровно один раз) определяется как ломаная L , которая строится путем выбора одной точки p_{k_i} из каждого кластера C_i :

$$L = [p_0, p_{k_1}, p_{k_2}, \dots, p_{k_N}, p_0], \quad \text{где } p_{k_i} \in C_i, \quad i \in \{1, \dots, N\}$$

- Длина ломаной L вычисляется как сумма расстояний между последовательными точками, включая возвращение в стартовую точку:

$$\text{length}(L) = d(p_0, p_{k_1}) + \sum_{i=2}^N d(p_{k_{i-1}}, p_{k_i}) + d(p_{k_N}, p_0)$$

- Цель задачи: минимизировать длину ломаной L :

$$\min_{p_{k_1} \in C_1, \dots, p_{k_N} \in C_N} \text{length}(L)$$

4 Описание использованных алгоритмов

4.1 Алгоритм полного перебора

Алгоритм полного перебора для данной задачи включает проверку всех возможных комбинаций выбора точек из каждого кластера и всех возможных перестановок их последовательности для построения замкнутого маршрута. Основные шаги алгоритма можно разбить следующим образом:

- **Выбор точек из кластеров**

Для каждого кластера C_i ($i \in \{1, \dots, N\}$) требуется выбрать одну точку p_{k_i} из множества точек кластера. Поскольку это необходимо сделать для каждого из N кластеров, общее число возможных комбинаций точек, которые могут быть выбраны, составляет

$$\prod_{i=1}^N |C_i| = \prod_{i=1}^N a_i$$

- **Перебор всех перестановок выбранных точек**

После того, как выбрана комбинация точек $\{p_{k_1}, p_{k_2}, \dots, p_{k_N}\}$, необходимо сгенерировать все возможные перестановки последовательности этих точек, чтобы учесть все возможные пути обхода кластеров. Число таких перестановок для N точек составляет $N!$.

- **Вычисление длины маршрута**

Для каждой перестановки вычисляется длина ломаной L по формуле:

$$\text{length}(L) = d(p_0, p_{k_1}) + \sum_{i=2}^N d(p_{k_{i-1}}, p_{k_i}) + d(p_{k_N}, p_0)$$

где $d(p_i, p_j)$ — евклидово расстояние между точками p_i и p_j .

- **Сравнение и выбор минимальной длины**

Из всех сгенерированных ломаных L (для всех возможных выборов точек из кластеров и перестановок) выбирается маршрут с минимальной длиной:

$$L_{\min} = \arg \min_L \text{length}(L)$$

Это решение является глобально оптимальным для задачи, так как алгоритм полного перебора перебирает все возможные варианты.

Формула для подсчёта полного количества различных маршрутов:

$$N! \cdot \prod_{i=1}^N a_i$$

Если же число точек в каждом кластере одинаковое, равное a , тогда формула принимает вид:

$$N! \cdot a^N$$

4.2 Простейшая эвристика: жадный алгоритм

1. Инициализация:

- $L := [p_0]$ – начальный путь, содержащий только стартовую точку.
- $V := \emptyset$ – множество индексов посещенных кластеров.
- $\check{p} := p_0$ – текущая точка пути.

2. Пока $|V| < N$ (пока не посещены все кластеры):

2.1 Найти $\mathring{p}$ – ближайшую точку к текущей точке $\check{p}$ среди точек, принадлежащих непосещенным кластерам:

$$\mathring{p} := \arg \min_{p_j \in \mathcal{P} \setminus \bigcup_{i \in V} C_i} d(\check{p}, p_j)$$

2.2 Определить индекс $\check{c}$ – ближайшего кластера, которому принадлежит точка $\mathring{p}$:

$$\check{c} := i, \quad \text{где } \mathring{p} \in C_i$$

2.3 Добавить точку $\mathring{p}$ в путь L :

$$L := L \cup [\mathring{p}]$$

2.4 Добавить индекс кластера $\check{c}$ ко множеству посещенных кластеров V :

$$V := V \cup \{\check{c}\}$$

2.5 Обновить текущую точку пути:

$$\check{p} := \mathring{p}$$

3. Замкнуть путь, добавив стартовую точку p_0 :

$$L := L \cup [p_0]$$

4. Вернуть полученный путь L .

4.3 Точный алгоритм на основе динамического программирования

Введём искусственный кластер C_0 , состоящий из одной единственной точки p_0 , из которой должен начинаться маршрут, и в котором он должен завершиться.

$\check{p}$ – точка, для которой нужно посчитать оптимальный маршрут, при заданном множестве непосещённых кластеров;

$\mathcal{S} := \{\tilde{C}_i\}_{i=\{1, \dots, N\}}$ – множество непосещённых кластеров;

Математическое описание функции g_{dist} , возвращающей длину оптимального маршрута обхода кластеров:

$$g_{dist}(\check{p}, \mathcal{S}) = \min_{\forall \tilde{C} \in \mathcal{S}, \forall p \in \tilde{C}} \left\{ d(\check{p}, p) + g_{dist}(p, \mathcal{S} \setminus \{\tilde{C}\}) \right\}$$

Зададим функцию $g(\check{p}, \mathcal{S})$ аналогичную g_{dist} , но которая, в отличие от неё, возвращает не только длину оптимального пути обхода кластеров, но и маршрут обхода, в виде следующего алгоритма (псевдокод):

1. **Вход:** $\check{p}$ – текущая точка, $\mathcal{S}$ – множество непосещённых кластеров.

2. **Если** $\mathcal{S} = \emptyset$:

2.1 **Вернуть** $\langle d(\check{p}, p_0), [p_0] \rangle$

3. **Иначе**

3.1 **Установить** $\text{min_distance} := \infty$

3.2 **Установить** $\text{optimal_route} := []$

3.3 **Для всех** $\tilde{C} \in \mathcal{S}$, **для всех** $p \in \tilde{C}$

3.3.1 **Вычислить** $\langle \text{distance}, \text{route} \rangle := g(p, \mathcal{S} \setminus \{\tilde{C}\})$

3.3.2 **Установить** $\text{total_distance} := d(\check{p}, p) + \text{distance}$

3.3.3 **Если** $\text{total_distance} < \text{min_distance}$:

3.3.3.1 **Установить** $\text{min_distance} := \text{total_distance}$

3.3.3.2 **Добавить** точку p следующей в маршруте route

3.3.3.3 **Установить** $\text{optimal_route} := \text{route}$

4. **Вернуть** $\langle \text{min_distance}, \text{optimal_route} \rangle$

Таким образом, чтобы получить решение поставленной задачи, необходимо вычислить значение $g(p_0, \mathcal{C})$.

Решение является глобально оптимальным из определения функции g .

Дополнительно отметим, что к алгоритму можно применить мемоизацию, так как функция $g(\check{p}, \mathcal{S})$ вызывается несколько раз внутри рекурсии в случаях, когда $|\mathcal{S}| \leq 1$. Данный приём позволит сократить время выполнения алгоритма за счёт использования уже подсчитанных значений.

5 Тестовый пример

5.1 Описание тестовой задачи

- Стартовая точка: $p_0 = (0, 0)$
- Кластеры, составляющие $\mathcal{C}$:
 - $C_1 = \{p_1 = (69, 221), p_2 = (134, 182)\};$
 - $C_2 = \{p_3 = (244, 230), p_4 = (235, 216)\};$
 - $C_3 = \{p_5 = (98, 356), p_6 = (101, 335), p_7 = (86, 332)\}.$

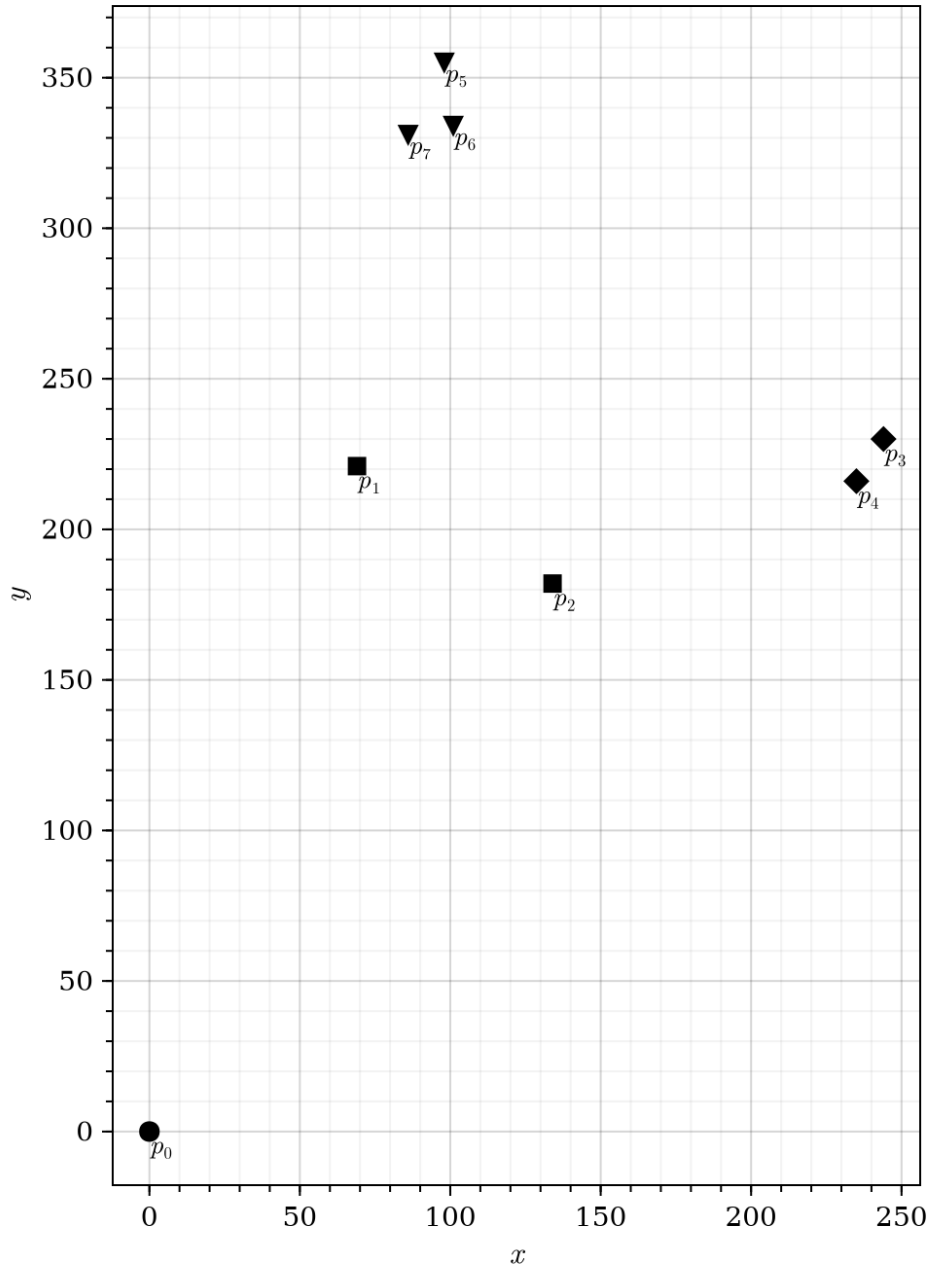


Рис. 2: Общий вид тестового множества точек; многоугольники – точки разных кластеров

Зная координаты точек, можно заранее вычислить матрицу расстояний

$$\mathbf{M}[i][j] := d(p_i, p_j) \forall i, j \in \{0, \dots, 7\},$$

исключая значения расстояний между точек одного кластера (эти расстояния не понадобятся):

		C ₀	C ₁		C ₂		C ₃		
		p ₀	p1	p2	p3	p4	p5	p6	p7
C ₀	p ₀	0	231.52	226.01	335.31	319.19	369.24	349.89	342.96
C ₁	p1	231.52	0	-	175.23	166.08	138.08	118.41	112.29
	p2	226.01	-	0	120.02	106.57	177.69	156.52	157.49
C ₂	p3	335.31	175.23	120.02	0	-	192.85	177.41	188.06
	p4	319.19	166.08	106.57	-	0	195.88	179.21	188.83
C ₃	p5	369.24	138.08	177.69	192.85	195.88	0	-	-
	p6	349.89	118.41	156.52	177.41	179.21	-	0	-
	p7	342.96	112.29	157.49	188.06	188.83	-	-	0

Рис. 3: Матрица расстояний $\mathbf{M}$ для тестового примера

5.2 Решение тестового примера жадным алгоритмом

1. Инициализация:

1.1 $L := [p_0]$ – список точек итоговой ломаной;

1.2 $V := \{ \}$ – индексы посещённых множеств;

1.3 $\check{p} := p_0$ – текущая точка пути.

Будем пользоваться предварительно сосчитанной матрицей расстояний $\mathbf{M}$.

2. $|V| = 0 < N = 3$ – остались не посещённые кластеры:

2.1 К текущей $\check{p}$ найдём ближайшую $\mathring{p} = p_i : i = \arg \min_{i \in \{1, \dots, K\} \setminus V} \{\mathbf{M}[0][i]\} = 2 \Rightarrow \Rightarrow \mathbf{M}[0][2] = 226.01$

2.2 Определяем, к какому кластеру принадлежит новая точка: $p_2 \in C_1 \Rightarrow \check{c} := 1$

2.3 Запоминаем индекс этого кластера: $V := V \cup \{\check{c}\} = \{1\}$

2.4 К точкам ломаной добавим новую: $L := L \cup [\mathring{p}] = L \cup [p_2] = [p_0, p_2]$

2.5 Обновим текущую точку: $\check{p} := \mathring{p} = p_2$

3. $|V| = 1 < N = 3$ – остались не посещённые кластеры:

3.1 К текущей $\check{p}$ найдём ближайшую $\mathring{p} = p_i : i = \arg \min_{i \in \{1, \dots, K\} \setminus V} \{\mathbf{M}[2][i]\} = 4 \Rightarrow \Rightarrow \mathbf{M}[2][4] = 106.57$

3.2 Определяем, к какому кластеру принадлежит новая точка: $p_4 \in C_2 \Rightarrow \check{c} := 2$

3.3 Запоминаем индекс этого кластера: $V := V \cup \{\check{c}\} = \{1, 2\}$

3.4 К точкам ломаной добавим новую: $L := L \cup [\overset{\circ}{p}] = L \cup [p_4] = [p_0, p_2, p_4]$

3.5 Обновим текущую точку: $\check{p} := \overset{\circ}{p} = p_4$

4. $|V| = 2 < N = 3$ – остались не посещённые кластеры:

4.1 К текущей $\check{p}$ найдём ближайшую $\overset{\circ}{p} = p_i : i = \arg \min_{i \in \{1, \dots, K\} \setminus V} \{\mathbf{M}[4][i]\} = 6 \implies \mathbf{M}[4][6] = 179.21$

4.2 Определяем, к какому кластеру принадлежит новая точка: $p_6 \in C_3 \implies \check{c} := 3$

4.3 Запоминаем индекс этого кластера: $V := V \cup \{\check{c}\} = \{1, 2, 3\}$

4.4 К точкам ломаной добавим новую: $L := L \cup [\overset{\circ}{p}] = L \cup [p_6] = [p_0, p_2, p_4, p_6]$

4.5 Обновим текущую точку: $\check{p} := \overset{\circ}{p} = p_6$

5. $|V| = 3 \not< N = 3$ – все кластеры посещены, замыкаем маршрут:

$$L := L \cup [p_0] = [\mathbf{p}_0, \mathbf{p}_2, \mathbf{p}_4, \mathbf{p}_6, \mathbf{p}_0]$$

Общая длина маршрута:

$$\text{length}(L) = \mathbf{M}[0][2] + \mathbf{M}[2][4] + \mathbf{M}[4][6] + \mathbf{M}[6][0] = \mathbf{861.7}$$

5.3 Решение тестового примера методом динамического программирования

Введём дополнительный кластер $C_0 = \{p_0 = (0, 0)\}$;

1. $\forall i \in 1..7 : g(\mathbf{p}_i, \emptyset) = (\mathbf{M}[i][0], [\mathbf{p}_0])$
2. $g(\mathbf{p}_3, \{\mathbf{C}_3\}) = \min\{ \mathbf{M}[3][5] + g(p_5, \emptyset) = 562.09, \mathbf{M}[3][6] + g(p_6, \emptyset) = 527.3, \mathbf{M}[3][7] + g(p_7, \emptyset) = 531.02 \} = \mathbf{527.3}$; оптимальный путь: $[\mathbf{p}_6, \mathbf{p}_0]$
3. $g(\mathbf{p}_4, \{\mathbf{C}_3\}) = \min\{ \mathbf{M}[4][5] + g(p_5, \emptyset) = 565.12, \mathbf{M}[4][6] + g(p_6, \emptyset) = 529.1, \mathbf{M}[4][7] + g(p_7, \emptyset) = 531.79 \} = \mathbf{529.1}$; оптимальный путь: $[\mathbf{p}_6, \mathbf{p}_0]$
4. $g(\mathbf{p}_5, \{\mathbf{C}_2\}) = \min\{ \mathbf{M}[5][3] + g(p_3, \emptyset) = 528.17, \mathbf{M}[5][4] + g(p_4, \emptyset) = 515.07 \} = \mathbf{515.07}$; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_0]$
5. $g(\mathbf{p}_6, \{\mathbf{C}_2\}) = \min\{ \mathbf{M}[6][3] + g(p_3, \emptyset) = 512.72, \mathbf{M}[6][4] + g(p_4, \emptyset) = 498.4 \} = \mathbf{498.4}$; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_0]$

$$\begin{aligned}
6. \quad g(\mathbf{p}_7, \{\mathbf{C}_2\}) &= \min\{ \mathbf{M}[7][3] + g(p_3, \emptyset) = 523.38, \\
&\mathbf{M}[7][4] + g(p_4, \emptyset) = 508.02 \} = \\
&= \mathbf{508.02}; \text{ оптимальный путь: } [\mathbf{p}_4, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
7. \quad g(\mathbf{p}_1, \{\mathbf{C}_2, \mathbf{C}_3\}) &= \min\{ \mathbf{M}[1][3] + g(p_3, \{C_3\}) = 702.53, \\
&\mathbf{M}[1][4] + g(p_4, \{C_3\}) = 695.18, \mathbf{M}[1][5] + g(p_5, \{C_2\}) = 653.15, \\
&\mathbf{M}[1][6] + g(p_6, \{C_2\}) = 616.81, \mathbf{M}[1][7] + g(p_7, \{C_2\}) = 620.31 \} = \\
&= \mathbf{616.81}; \text{ оптимальный путь: } [\mathbf{p}_6, \mathbf{p}_4, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
8. \quad g(\mathbf{p}_2, \{\mathbf{C}_2, \mathbf{C}_3\}) &= \min\{ \mathbf{M}[2][3] + g(p_3, \{C_3\}) = 647.32, \\
&\mathbf{M}[2][4] + g(p_4, \{C_3\}) = 635.68, \mathbf{M}[2][5] + g(p_5, \{C_2\}) = 692.75, \\
&\mathbf{M}[2][6] + g(p_6, \{C_2\}) = 654.92, \mathbf{M}[2][7] + g(p_7, \{C_2\}) = 665.51 \} = \\
&= \mathbf{635.68}; \text{ оптимальный путь: } [\mathbf{p}_4, \mathbf{p}_6, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
9. \quad g(\mathbf{p}_1, \{\mathbf{C}_3\}) &= \min\{ \mathbf{M}[1][5] + g(p_5, \emptyset) = 507.32, \\
&\mathbf{M}[1][6] + g(p_6, \emptyset) = 468.3, \mathbf{M}[1][7] + g(p_7, \emptyset) = 455.25 \} = \\
&= \mathbf{455.25}; \text{ оптимальный путь: } [\mathbf{p}_7, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
10. \quad g(\mathbf{p}_2, \{\mathbf{C}_3\}) &= \min\{ \mathbf{M}[2][5] + g(p_5, \emptyset) = 546.93, \\
&\mathbf{M}[2][6] + g(p_6, \emptyset) = 506.41, \\
&\mathbf{M}[2][7] + g(p_7, \emptyset) = 500.45 \} = \\
&= \mathbf{500.45}; \text{ оптимальный путь: } [\mathbf{p}_7, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
11. \quad g(\mathbf{p}_5, \{\mathbf{C}_1\}) &= \min\{ \mathbf{M}[5][1] + g(p_1, \emptyset) = 369.6, \\
&\mathbf{M}[5][2] + g(p_2, \emptyset) = 403.69 \} = \\
&= \mathbf{369.6}; \text{ оптимальный путь: } [\mathbf{p}_1, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
12. \quad g(\mathbf{p}_6, \{\mathbf{C}_1\}) &= \min\{ \mathbf{M}[6][1] + g(p_1, \emptyset) = 349.93, \\
&\mathbf{M}[6][2] + g(p_2, \emptyset) = 382.53 \} = \\
&= \mathbf{349.93}; \text{ оптимальный путь: } [\mathbf{p}_1, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
13. \quad g(\mathbf{p}_7, \{\mathbf{C}_1\}) &= \min\{ \mathbf{M}[7][1] + g(p_1, \emptyset) = 343.82, \\
&\mathbf{M}[7][2] + g(p_2, \emptyset) = 383.5 \} = \\
&= \mathbf{343.82}; \text{ оптимальный путь: } [\mathbf{p}_1, \mathbf{p}_0]
\end{aligned}$$

$$\begin{aligned}
14. \quad g(\mathbf{p}_3, \{\mathbf{C}_1, \mathbf{C}_3\}) &= \min\{ \mathbf{M}[3][1] + g(p_1, \{C_3\}) = 630.48, \\
&\mathbf{M}[3][2] + g(p_2, \{C_3\}) = 620.47, \mathbf{M}[3][5] + g(p_5, \{C_1\}) = 562.45, \\
&\mathbf{M}[3][6] + g(p_6, \{C_1\}) = 527.34, \mathbf{M}[3][7] + g(p_7, \{C_1\}) = 531.88 \} =
\end{aligned}$$

= **527.34**; оптимальный путь: $[\mathbf{p}_6, \mathbf{p}_1, \mathbf{p}_0]$

15. $g(\mathbf{p}_4, \{\mathbf{C}_1, \mathbf{C}_3\}) = \min\{ \mathbf{M}[4][1] + g(p_1, \{C_3\}) = 621.33,$
 $\mathbf{M}[4][2] + g(p_2, \{C_3\}) = 607.02, \mathbf{M}[4][5] + g(p_5, \{C_1\}) = 565.48,$
 $\mathbf{M}[4][6] + g(p_6, \{C_1\}) = 529.14, \mathbf{M}[4][7] + g(p_7, \{C_1\}) = 532.65\} =$
= **529.14**; оптимальный путь: $[\mathbf{p}_6, \mathbf{p}_1, \mathbf{p}_0]$

16. $g(\mathbf{p}_1, \{\mathbf{C}_2\}) = \min\{ \mathbf{M}[1][3] + g(p_3, \emptyset) = 510.55,$
 $\mathbf{M}[1][4] + g(p_4, \emptyset) = 485.26\} =$
= **485.26**; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_0]$

17. $g(\mathbf{p}_2, \{\mathbf{C}_2\}) = \min\{ \mathbf{M}[2][3] + g(p_3, \emptyset) = 455.33,$
 $\mathbf{M}[2][4] + g(p_4, \emptyset) = 425.76\} =$
= **425.76**; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_0]$

18. $g(\mathbf{p}_3, \{\mathbf{C}_1\}) = \min\{ \mathbf{M}[3][1] + g(p_1, \emptyset) = 406.75,$
 $\mathbf{M}[3][2] + g(p_2, \emptyset) = 346.03\} =$
= **346.03**; оптимальный путь: $[\mathbf{p}_2, \mathbf{p}_0]$

19. $g(\mathbf{p}_4, \{\mathbf{C}_1\}) = \min\{ \mathbf{M}[4][1] + g(p_1, \emptyset) = 397.6,$
 $\mathbf{M}[4][2] + g(p_2, \emptyset) = 332.58\} =$
= **332.58**; оптимальный путь: $[\mathbf{p}_2, \mathbf{p}_0]$

20. $g(\mathbf{p}_5, \{\mathbf{C}_1, \mathbf{C}_2\}) = \min\{ \mathbf{M}[5][1] + g(p_1, \{C_2\}) = 623.34,$
 $\mathbf{M}[5][2] + g(p_2, \{C_2\}) = 603.44, \mathbf{M}[5][3] + g(p_3, \{C_1\}) = 538.88,$
 $\mathbf{M}[5][4] + g(p_4, \{C_1\}) = 528.46\} =$
= **528.46**; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_2, \mathbf{p}_0]$

21. $g(\mathbf{p}_6, \{\mathbf{C}_1, \mathbf{C}_2\}) = \min\{ \mathbf{M}[6][1] + g(p_1, \{C_2\}) = 603.67,$
 $\mathbf{M}[6][2] + g(p_2, \{C_2\}) = 582.28, \mathbf{M}[6][3] + g(p_3, \{C_1\}) = 523.43,$
 $\mathbf{M}[6][4] + g(p_4, \{C_1\}) = 511.79\} =$
= **511.79**; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_2, \mathbf{p}_0]$

22. $g(\mathbf{p}_7, \{\mathbf{C}_1, \mathbf{C}_2\}) = \min\{ \mathbf{M}[7][1] + g(p_1, \{C_2\}) = 597.56,$
 $\mathbf{M}[7][2] + g(p_2, \{C_2\}) = 583.25, \mathbf{M}[7][3] + g(p_3, \{C_1\}) = 534.09,$
 $\mathbf{M}[7][4] + g(p_4, \{C_1\}) = 521.41\} =$
= **521.41**; оптимальный путь: $[\mathbf{p}_4, \mathbf{p}_2, \mathbf{p}_0]$

23. $g(\mathbf{p}_0, \{\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3\}) = \min\{ \mathbf{M}[0][1] + g(p_1, \{C_2, C_3\}) = 848.33,$
 $\mathbf{M}[0][2] + g(p_2, \{C_2, C_3\}) = 861.68, \mathbf{M}[0][3] + g(p_3, \{C_1, C_3\}) = 862.65,$
 $\mathbf{M}[0][4] + g(p_4, \{C_1, C_3\}) = 848.33, \mathbf{M}[0][5] + g(p_5, \{C_1, C_2\}) = 897.7,$
 $\mathbf{M}[0][6] + g(p_6, \{C_1, C_2\}) = 861.68, \mathbf{M}[0][7] + g(p_7, \{C_1, C_2\}) = 864.37\} =$
 $= 848.33$; оптимальный путь: $[\mathbf{p}_1, \mathbf{p}_6, \mathbf{p}_4, \mathbf{p}_0]$

Ответ: длина оптимального пути: **848.33**; оптимальный маршрут, включая стартовую точку: $[\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_6, \mathbf{p}_4, \mathbf{p}_0]$.

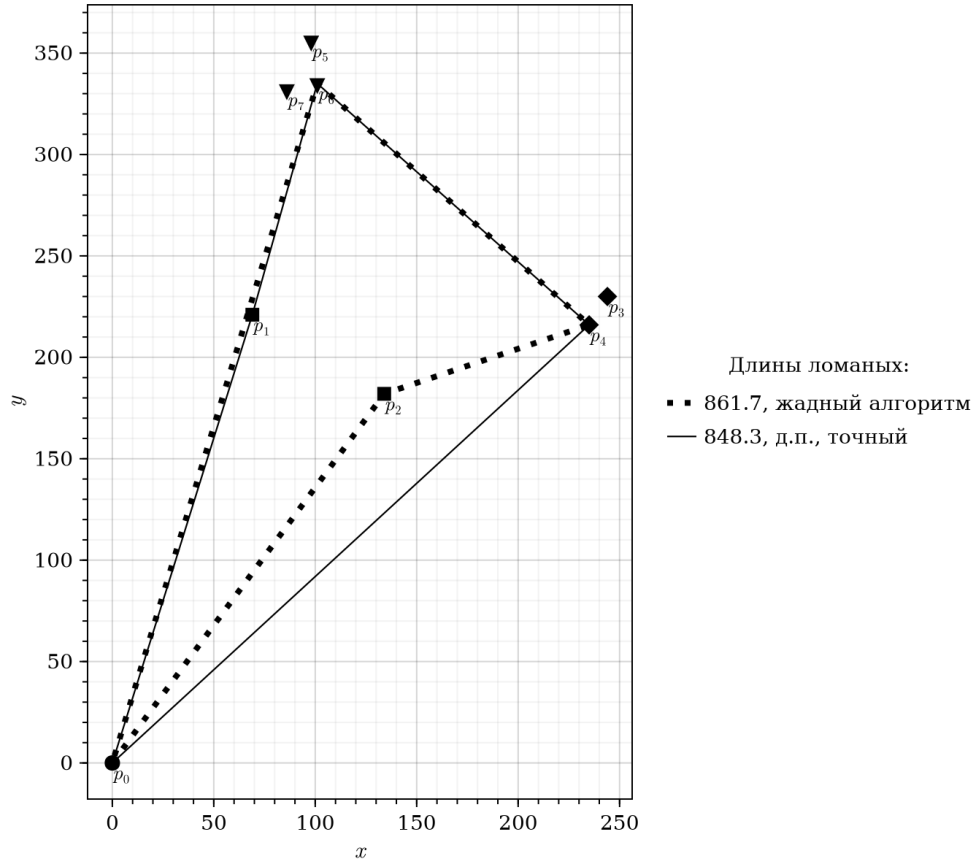


Рис. 4: Визуальное сравнение работы двух алгоритмов для тестового примера

$$L_{\text{ж}} = [\mathbf{p}_0, \mathbf{p}_2, \mathbf{p}_4, \mathbf{p}_6, \mathbf{p}_0]$$

$$L_{\text{д.п.}} = [\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_6, \mathbf{p}_4, \mathbf{p}_0]$$

6 Сравнение методов решения обобщенной задачи коммивояжёра

По результатам численного эксперимента по сравнению работы алгоритмов на разном количестве множеств и разным количеством точек в каждом были получены следующие графики:

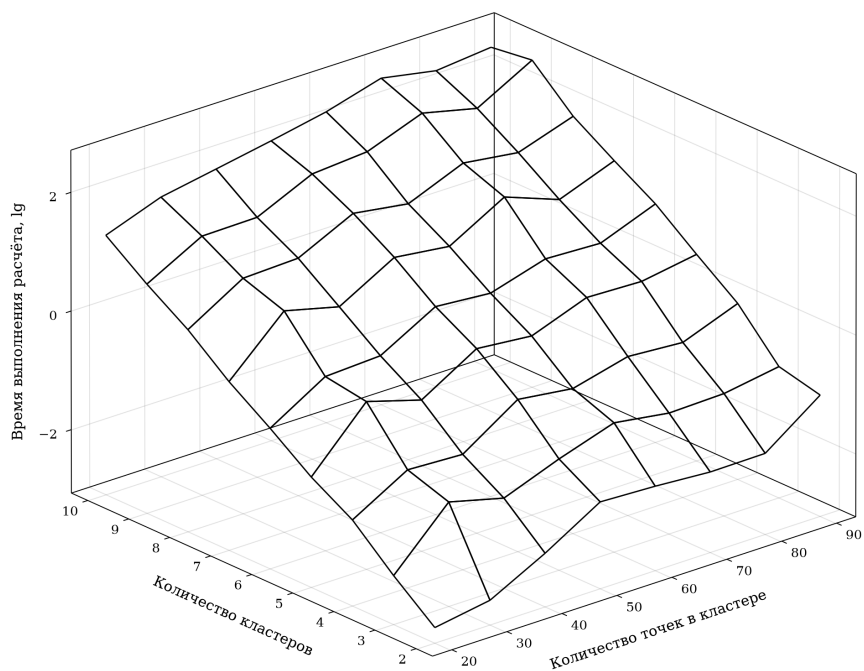


Рис. 5: Время выполнения алгоритма динамического программирования (ось Z логарифмическая)

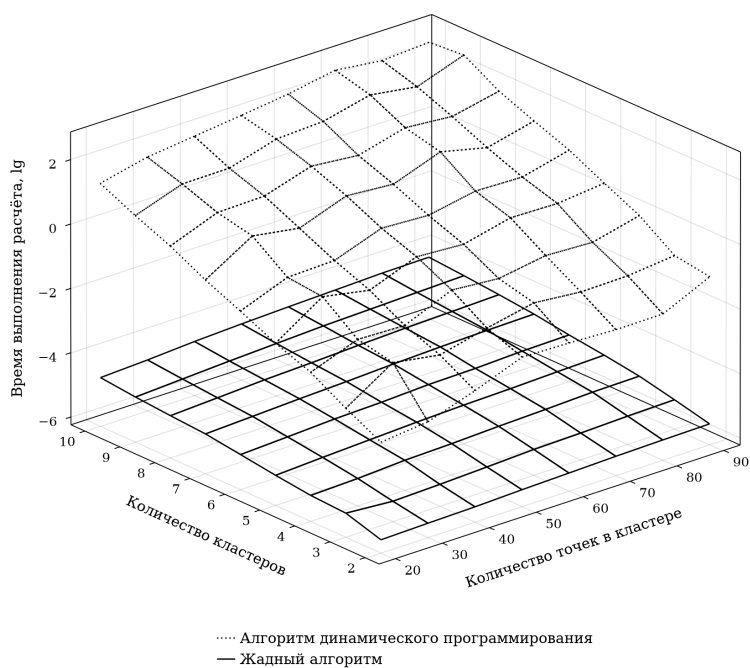


Рис. 6: Сравнение времени выполнения жадного алгоритма и алгоритма, основанного на д.п. (ось Z логарифмическая)

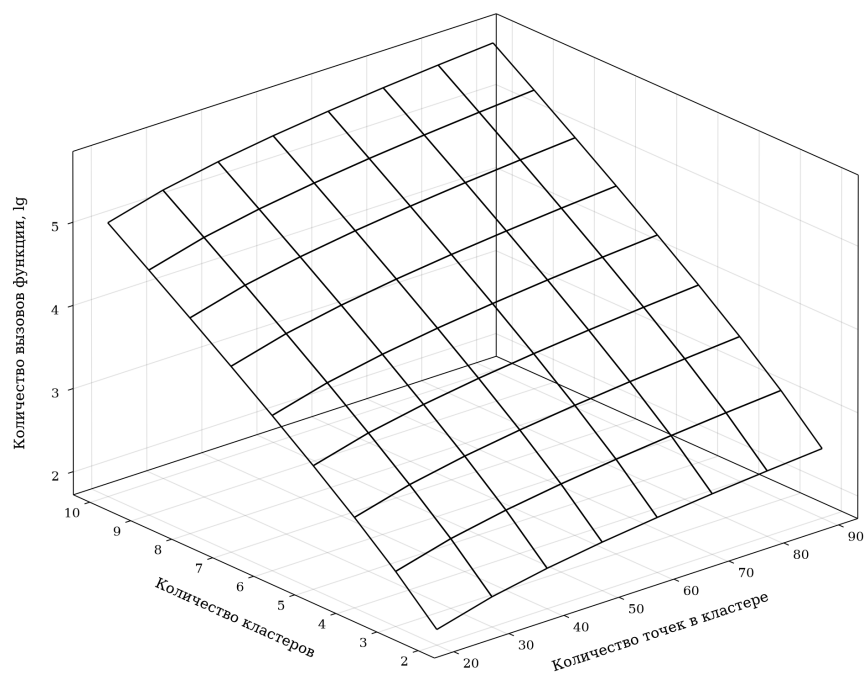


Рис. 7: Количество вызовов функции g в решении методом динамического программирования (ось Z логарифмическая)

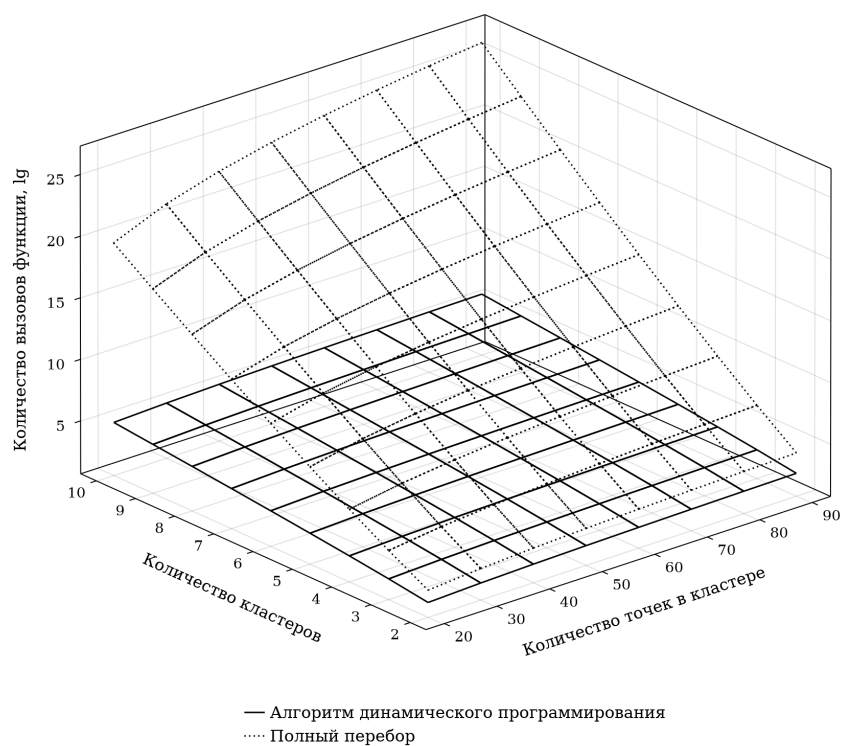


Рис. 8: Сравнение количества вызовов функции g в решении методом динамического программирования с количеством вариантов при полном переборе (значения для полного перебора получены теоретически, в соответствии с формулами из пар. 4.1).

7 Заключение

7.1 Заключение

В ходе данной работы была рассмотрена обобщенная задача коммивояжера (GTSP) в контексте оптимизации маршрута лазерной головки в станках с ЧПУ; были перечислены различные подходы к её решению.

Методы были формализованы и применены к тестовому примеру жадный алгоритм и алгоритм, основанный на динамическом программировании. Был проведён сравнительный анализ этих алгоритмов с теоретическими оценками для алгоритма полного перебора, что позволило провести практическое сравнение их эффективности.

Анализ показал:

- Метод полного перебора гарантирует нахождение оптимального решения, но становится неприменимым для задач большой размерности из-за сверхэкспоненциального роста времени выполнения.
- Метод динамического программирования обеспечивает оптимальное решение и значительно сокращает время вычислений по сравнению с полным перебором, особенно для задач средней размерности.
- Жадный алгоритм предоставляет самое быстрое решение, но не гарантирует оптимальности. Он может быть полезен для получения начального приближения или в ситуациях, где скорость важнее точности.

7.2 Вывод

Проведенное исследование позволяет сделать следующие выводы:

- Для малых задач (до 10 кластеров) метод динамического программирования является хорошим выбором, обеспечивая точное решение за приемлемое время;
- Для задач большой размерности рекомендуется использовать эвристические и метаэвристические методы и комбинированные подходы;
- На практике, выбор метода оптимизации работы станка с ЧПУ должен основываться на экономической целесообразности, соотнося затраты на вычисление и издержки, которые следуют за использованием неоптимального маршрута.

Список литературы

- [1] M. El Krari. “A pre-processing reduction method for the generalized travelling salesman problem”. B: (2021). URL: <https://link.springer.com/article/10.1007/s12351-019-00533-w>.
- [2] Gregory Gutin и Daniel Karapetyan. “Generalized Traveling Salesman Problem Reduction Algorithms”. B: (2009). URL: https://www.erudit.org/en/journals/aor/2009-v4-n2-aor4_2/aor4_2art06/.
- [3] Charles E. Noon и James C. Bean. “An Efficient Transformation Of The Generalized Traveling Salesman Problem”. B: 31.1 (1993), с. 39—44. URL: <https://doi.org/10.1080/03155986.1993.11732212>.
- [4] Alexander Petunin, E. Polishchuk и S. Ukolov. “A NEW ALGORITHM FOR CONSTRUCTING THE SHORTEST TOUR OF A FINITE SET OF DISJOINT CONTOURS ON A PLANE”. B: (2021). DOI: 10.18522/2311-3103-2021-1-149-165.
- [5] Petrică C. Pop и др. “A comprehensive survey on the generalized traveling salesman problem”. B: (2024). URL: <https://www.sciencedirect.com/science/article/pii/S0377221723005581>.
- [6] Milan Milosavljević Vladimir Dimitrijević и Milan Marković. “A BRANCH AND BOUND ALGORITHM FOR SOLVING A GENERALIZED TRAVELING SALESMAN PROBLEM”. B: (1996). URL: <https://www.jstor.org/stable/43666465>.
- [7] Benjamin W.-S. Wah Yao-Nan Lien Eva Ma. “Transformation of the generalized traveling-salesman problem into the standard traveling-salesman problem”. B: (1993). URL: <https://www.sciencedirect.com/science/article/abs/pii/0020025593901337>.